Vectra: A High-Performance Geo-Distributed Vector Search and Transformer Inference Engine

Project Plan and Technical Design Document

A bespoke geo-distributed retrieval and inference platform for real-time property price prediction

Project window	January 6, 2025 – May 30, 2025
Document status	Approved baseline (v1.0)
Prepared for	Real-Time Property Intelligence Platform
Document purpose	Define the objectives, architecture, execution plan, and acceptance criteria for replacing managed backend services with a bespoke geo-distributed C++ vector search engine and an automatically optimized Transformer inference path.

Table of Contents

Right-click the table and choose "Update Field" to refresh page numbers.

Table of Contents 2
1. Executive Summary 3
2. Project Objectives and Success Metrics 3
3. Background and Problem Statement 4
3.1 Why managed backend services cannot meet the retrieval target 4
3.2 Why eager PyTorch cannot meet the inference target 5
4. System Architecture Overview 6
5. Workstream A — Geo-Distributed C++ Vector Search Engine 7
5.1 Asynchronous gRPC transport on a thread-per-core model 7
5.2 GeoHash-based spatial sharding and deterministic routing 8
5.3 Flat-memory HNSW indexing with product quantization 8
6. Workstream B — Custom Fused Triton Kernels for Geo-Spatial Scoring 9
6.1 Baseline analysis: the spatial scorer is memory-bound in eager execution 9
6.2 Algebraic reformulation: turning distance computation into tensor-core GEMM 10
6.3 Tiled IO-aware execution and the HBM traffic budget 10
6.4 Why Triton rather than hand-written CUDA 11
7. Workstream C — Automated Compiler Pass for Kernel Substitution 11
7.1 Graph capture with TorchDynamo and torch.fx 11
7.2 Pattern definition and replacement via subgraph rewriting 12
7.3 Raising achieved memory bandwidth utilization from 40% to 65% 13
8. Project Timeline and Milestones 13
9. Benchmarking and Verification Methodology 14
10. Risk Register and Mitigations 15
11. Team, Resources, and Deliverables 16
12. Works Cited 16

1. Executive Summary

Real-time property price prediction at national scale sits at the intersection of two demanding disciplines: large-scale similarity retrieval over millions of embedded property records, and low-latency deep learning inference over the retrieved candidates. Managed backend services — hosted vector databases and general-purpose data platforms — are engineered for durability and generality rather than for the extreme read performance this product requires. Industry surveys of vector database management systems document that mainstream offerings are built around generalized sharding, scatter–gather query execution, and abstraction layers that add latency precisely where this platform can least afford it [1]. Project Vectra therefore replaces the managed backend with a purpose-built, geo-distributed C++ vector search engine, and pairs it with a GPU inference path in which the dominant spatial-scoring computation is executed by hand-written fused kernels rather than by eager-mode PyTorch [2].

The project commits to three quantitative outcomes. First, the search engine will sustain **50,000 requests per second** against a corpus of **10 million property embeddings** with **sub-millisecond p50 latency**, using custom GeoHash-based sharding and an asynchronous gRPC transport instead of a managed service. Second, custom fused Triton kernels for geo-spatial vector scoring will cut High Bandwidth Memory (HBM) traffic by **40% in bytes moved per query** and **triple inference throughput** relative to the eager PyTorch baseline, by explicitly optimizing global memory access patterns in the style of IO-aware attention kernels [3]. Third, a custom compiler pass operating on the torch.fx graph will automatically detect and replace default PyTorch operators with those fused kernels, raising achieved-versus-peak memory bandwidth utilization in Transformer price-prediction serving from **40% to 65%**. Every figure in this document is tied to a verification protocol defined in Section 9, and every external technique cited here is drawn from literature published no later than May 2025, the close of the project window.

The remainder of this document states the measurable objectives, motivates the design against published evidence, specifies the three workstreams, lays out the January–May 2025 execution schedule, and defines how each success metric will be measured and accepted. The document is written so that an engineering organization can execute it directly: each workstream section closes with concrete deliverables and the exact acceptance test that proves the corresponding headline metric.

2. Project Objectives and Success Metrics

The project is organized around three engineering workstreams, each owning one headline metric. The metrics are stated as they will appear in the final engineering record, together with the acceptance test that will certify them. All three must hold simultaneously in the production environment by the end of May 2025.

#	Headline achievement (final record wording)	Acceptance test
O1	Designed a geo-distributed C++ vector search engine with custom sharding and gRPC, replacing managed backend services to support 50,000 requests/second with sub-millisecond p50 latency for 10M properties.	30-minute sustained load test at 50,000 QPS against the production 10M-property index; p50 end-to-end retrieval latency < 1 ms; recall@10 ≥ 0.95 against brute-force ground truth, following ANN-Benchmarks evaluation practice [4].
O2	Programmed custom fused Triton kernels for geo-spatial vector scoring by optimizing global memory access patterns to cut HBM traffic by 40% (bytes moved per query) while tripling inference throughput compared to PyTorch.	Nsight Compute byte counters (dram__bytes_read + dram__bytes_write) per query: fused kernel ≤ 60% of eager baseline; serving throughput ≥ 3.0× eager PyTorch at matched numerics (allclose, rtol 1e-3) [3] [5].
O3	Developed a custom compiler pass to auto-detect and replace default PyTorch operators with optimized fused kernels, raising achieved-vs-peak memory bandwidth utilization from 40% to 65% in Transformer price-prediction serving.	With the pass enabled, zero graph breaks around the spatial-scoring region; Nsight Compute "Memory Throughput [% of peak]" for the serving kernel ≥ 65% versus the 40% baseline profile [6] [5].

Objective O1 targets the retrieval tier. The 50,000 QPS and sub-millisecond p50 combination is deliberately set beyond what generalized managed vector services advertise for this corpus class; published benchmarks show that in-memory graph indexes reach microsecond-range query latency on million-to-billion-scale corpora only when the index is fully RAM-resident and the query path is free of coordination overhead [4] [7]. Objective O2 targets the single most expensive computation in the pricing model — dense geo-spatial scoring between a query property and its retrieved candidates — which is memory-bound in eager execution and therefore leaves most of the GPU's arithmetic capacity idle [3] [8]. Objective O3 makes the optimization self-applying: data scientists keep writing ordinary PyTorch, and the deployment compiler rewrites their code into the fast path, preserving both performance and developer velocity [9] [10].

Each objective is decomposed into workstream-level deliverables in Sections 5 through 7, and each deliverable carries a milestone date in Section 8. The three objectives share one dependency: the fused kernel of O2 is the replacement target installed by the compiler pass of O3, so kernel correctness gates the compiler work, and the whole system is validated end to end only in the final month of the project.

3. Background and Problem Statement

3.1 Why managed backend services cannot meet the retrieval target

Generalized vector database management systems are designed to serve many workloads acceptably rather than one workload optimally. The standard architecture of these systems partitions a collection into shards and answers a query by scattering it to the relevant partitions and gathering partial results into a merged response; the canonical survey of the field notes that replication and sharding introduce "extra overhead for search queries due to scatter-gather," and that plan selection in such systems is rule- or cost-based machinery tuned for generality [1]. Purpose-built managed systems such as Milvus illustrate the same design center: LSM-style segment ingestion, replica-based availability, and cost-based plan selection optimized for operability across many workloads rather than for one ultra-low-latency query path [11]. When an index cannot remain memory-resident, traversal degenerates from a CPU-bound to an I/O-bound operation, and performance falls non-linearly as concurrency rises — an effect documented across disk-resident and compressed index studies and one of the central findings of the billion-scale ANN literature [7] [12].

Tail behavior compounds the structural cost. In any fan-out design the latency of one user request is bounded by the slowest of the shards it touches, so even rare millisecond-level stalls — garbage-collection pauses in managed runtimes, compaction, connection re-establishment — dominate the percentile profile at 50,000 requests per second; this is the classic "tail at scale" argument articulated by Dean and Barroso, who show that latency variability from shared resources and background daemons is amplified exactly in proportion to the number of subsystems a request must wait on [13]. A service written in a systems-level language with deterministic memory management, a non-blocking network path, and a routing scheme that sends each query to the one server that owns its GeoHash cell, and near a shard boundary to at most the owners of the nine surrounding cells, removes most of those amplification points by construction, which is the premise of Workstream A.

3.2 Why eager PyTorch cannot meet the inference target

Eager-mode PyTorch executes operators one at a time, materializing every intermediate tensor to HBM [2]. For the spatial-scoring layer of the pricing Transformer — pairwise distance computation followed by a kernel-shaped scoring function over an N×N candidate matrix — this means writing and re-reading large intermediate tensors between every arithmetic step. FlashAttention's IO analysis quantified the cost of this pattern: attention in eager form is bounded not by arithmetic but by reads and writes between HBM and on-chip SRAM, and the fix is to fuse the whole computation into a single kernel pass that never materializes the N×N intermediates [3]. The same analysis applies directly to distance-based scoring, because the RBF/exp-of-negative-squared-distance score has the same softmax-like reduction structure as attention [14] [15].

The hardware context makes the bottleneck concrete. An NVIDIA H100 SXM5 pairs enormous arithmetic throughput — on the order of 10^3 TFLOP/s of FP16/BF16 tensor-core compute — with an HBM3 subsystem of roughly 3.35 TB/s, so any kernel whose arithmetic

intensity is low is throttled by memory long before compute is saturated [16]. Transformer inference research reaches the same conclusion from the serving side: Pope et al. show that at low batch sizes inference cost is dominated by memory movement rather than FLOPs, and that raising achieved memory-bandwidth utilization is the primary lever for latency-bound serving [8]. Workstream B attacks the bandwidth wall directly with a fused Triton kernel, and Workstream C makes the attack automatic at deploy time.

4. System Architecture Overview

The production system has two tiers connected by an internal RPC boundary. The retrieval tier is a cluster of bare-metal C++ nodes, each owning a deterministic slice of the 10 million property embeddings; the inference tier is a GPU service running the Transformer pricing model. A request enters at a stateless routing layer, which computes the query's GeoHash cell, selects the retrieval node that owns it (plus the owners of any adjacent cells the search radius reaches), forwards the lookup over a pooled asynchronous gRPC channel, and streams the returned candidate embeddings to the inference service. The inference service scores the candidates against the query and returns a price estimate. The end-to-end budget is dominated by the two computations the project optimizes: sub-millisecond approximate nearest neighbor search inside one shard, and bandwidth-saturated spatial scoring on the GPU.

Tier	Component	Technology	Governing decisions
Edge	Routing layer	C++20 router library (engine/client) and its mirror in the inference gateway, driven by one shared configuration	GeoHash cell computation; single-shard routing; adjacent-cell duplication only when the search radius crosses a cell boundary [17] [18]
Transport	RPC fabric	gRPC C++ async API, HTTP/2	One CompletionQueue per core; thread-per-core pinning; pre-warmed channel pool to stay under HTTP/2 stream limits [19] [20]
Retrieval	Vector index	In-memory HNSW, flat array layout, PQ codes	Integer-offset graph in flat, fixed-stride arrays; PQ-coded traversal with an exact FP32 re-rank; SIMD distance kernels; recall tuned via ef_search [21] [22] [23]
Inference	Pricing model	Transformer with distance-based spatial scoring	RBF-form attention over retrieved candidates; compiled with torch.compile [14] [24]
Inference	Scoring kernel	OpenAI Triton	Euclidean-expansion GEMM on tensor cores; online softmax accumulators in registers; single-pass HBM traffic [3] [25] [26]

Deployment	Compiler integration	torch.fx / TorchInductor custom pass	subgraph_rewriter pattern matching; custom pass at TorchInductor's joint-graph pre-pass; no manual kernel calls in modeling code [9] [10]

Two architectural invariants protect the latency budget everywhere else. First, no request ever waits on a consensus protocol, a compaction, or a second shard in the common case: GeoHash prefix routing makes the common case a single network hop to a single in-memory index, and the adjacent-cell mechanism handles boundary correctness with a bounded, known fan-out of at most nine cells [17] [18]. Second, no intermediate tensor of the spatial-scoring computation ever touches HBM: the fused kernel reads each input embedding once and writes each output score once, so the scorer moves about 1.4% of the eager pipeline's bytes at the serving shape; it remains bandwidth-bound, but on a small fraction of the traffic [3]. The remainder of the document specifies each workstream in detail.

5. Workstream A — Geo-Distributed C++ Vector Search Engine

5.1 Asynchronous gRPC transport on a thread-per-core model

The network fabric uses gRPC's C++ asynchronous API rather than the ergonomic synchronous threading model. In the synchronous model the runtime multiplexes requests over an internal thread pool; at 50,000 concurrent request arrivals per second the operating system scheduler spends a growing fraction of every millisecond context-switching blocked threads, invalidating L1/L2 caches and converting CPU capacity into overhead. The asynchronous API inverts control: the application binds a CompletionQueue to the RPC machinery, presents every operation with a unique void* tag, and drives a poll loop on CompletionQueue::Next, so a thread never blocks inside the runtime [19]. The official tutorial's CallData pattern — a per-RPC state machine whose own memory address serves as the tag, with a fresh CallData spawned before the current one is processed — is adopted verbatim as the server's request lifecycle [19].

The deployment configuration dedicates exactly one polling thread and one completion queue per physical core, pinned, which yields a thread-per-core execution model with no oversubscription and deterministic, microsecond-scale dispatch [19]. gRPC's performance guide reports that two polling threads per completion queue performed best in its benchmarks [27], so the threads-per-queue ratio is a configuration parameter (ServerOptions.threads_per_queue) settled by the A1 load test rather than fixed. On the client side, a single gRPC channel maps to a single HTTP/2 connection whose concurrency is capped by the protocol's maximum concurrent streams setting [20]; pushing 50,000 QPS through one channel would silently queue requests inside the runtime and erase the server's async advantage. The routing layer therefore maintains a pre-warmed pool of channels per destination node and selects among them round-robin, keeping per-channel stream counts far below the negotiated limit [27] [20]. Deliverable A1 is the load-tested transport: a harness demonstrating

50,000 QPS of no-op RPCs through the pool with p99 dispatch overhead under 100 µs, measured per the methodology of Section 9.

5.2 GeoHash-based spatial sharding and deterministic routing

The 10 million property embeddings are partitioned with GeoHash, the hierarchical spatial encoding introduced by Niemeyer in 2008, which interleaves the bisected bits of latitude and longitude along a Z-order curve and renders the result as a base-32 string; each additional character multiplies spatial precision by 32, and physically adjacent points usually share a string prefix [17]. The system exploits prefix locality at three fixed precisions, using the standard cell dimensions published for the encoding [18].

GeoHash length	Approximate cell dimensions	Role in Vectra
3 characters	~156 km × 156 km	Macro-region: selects the geographic cluster / data center
4 characters	~39.1 km × 19.5 km	Primary shard key: maps each cell to exactly one owning C++ node via a deterministic routing table
5 characters	~4.9 km × 4.9 km	In-node inverted pre-filter applied before graph traversal
6 characters	~1.2 km × 0.6 km	Micro-neighborhood bound used for economic isolation in pricing features

Because GeoHash is a rigid grid, two properties can be physically adjacent yet fall on opposite sides of a bisecting line with different prefixes [17]. The router guarantees recall by computing the eight cells adjacent to the target cell and duplicating the query only to the shards governing cells that the requested search radius actually intersects [18]. The common case — a radius contained in one 39 km cell — touches exactly one server; the worst case touches nine, a bound that is known at route time and therefore budgetable, in contrast to the all-shard fan-out of generalized systems [1]. Deliverable A2 is the sharding service with its routing table, adjacency expansion, and a correctness suite that brute-force-verifies recall@10 ≥ 0.99 over randomly sampled radii drawn from production query distributions.

5.3 Flat-memory HNSW indexing with product quantization

Inside each shard the engine serves approximate nearest neighbor search with Hierarchical Navigable Small World graphs. HNSW builds a multi-layer proximity graph in which the bottom layer holds every vector and each upper layer holds an exponentially thinned subset connected by long-range links; a query enters at the top, greedily descends to the target neighborhood, and resolves the final neighbors at layer 0, giving logarithmic search complexity with state-of-the-art recall-latency trade-offs [21]. The two construction-time knobs — M, the maximum bi-directional degree per node, and ef_construction, the candidate-list size during insertion — together with the runtime knob ef_search provide explicit control over the

recall/latency operating point, which the project tunes to hold recall@10 ≥ 0.95 while keeping per-query traversal in the microsecond range [21] [4].

The implementation departs from textbook object-oriented HNSW in its memory layout. Pointer-chasing graphs destroy sub-millisecond latency because every hop is a cache miss; the reference C++ implementations that dominate public benchmarks instead store the graph in flat contiguous arrays with integer offsets in place of pointers, so that fetching a node pulls its topological neighborhood into cache lines with it, and distance evaluation vectorizes cleanly with SIMD [23] [28]. Vectra adopts the same arena layout: one contiguous block per layer, edges encoded as 32-bit offsets, and each node's PQ code stored inside its layer-0 record, so that a traversal step fetches the data of its next distance computation together with the adjacency list. Capacity is protected by product quantization — the classical 8-bit-subvector codebook scheme of Jégou et al. compresses a 128-dimensional FP32 embedding 32× (512 bytes to 16 bytes at 16 subspaces), and graph traversal runs on these codes [22]. PQ distances alone cannot meet the recall target: ranking by them caps recall@10 near 0.65 on the property-embedding corpus, so the best candidates of every search are re-ranked with exact FP32 distances and the FP32 vectors stay resident. The index is about 785 bytes per vector, roughly 7.9 GB for the 10M corpus, still locked in RAM with the traversal entirely CPU-bound. Compressed-domain search practice from FAISS, Google's ScaNN, and the LVQ design behind Intel's SVS library confirm that quantization plus careful data layout, not algorithmic novelty, is what separates benchmark-leading systems from naive ones [29] [30] [31]. Deliverable A3 is the shard service passing the O1 load test: 50,000 QPS across the cluster, p50 end-to-end latency under 1 ms, recall@10 ≥ 0.95 against brute force.

6. Workstream B — Custom Fused Triton Kernels for Geo-Spatial Scoring

6.1 Baseline analysis: the spatial scorer is memory-bound in eager execution

The pricing Transformer scores a query property against retrieved candidates with a radial-basis spatial interaction: the score between query embedding q and candidate embedding k is proportional to $\exp(-\|q - k\|^2 / 2\sigma^2)$, normalized softmax-style across candidates. This is a legitimate attention variant — the kernel view of attention shows that standard attention is itself an RBF kernel over exp of the inner product, and distance-based attention is a studied alternative [14] [15]. In eager PyTorch the computation decomposes into a subtraction broadcast, an elementwise square, a reduction, an exponential, and a softmax — each a separate kernel launch that materializes its result in HBM: the subtraction and the square produce an N×N×D tensor, the later steps N×N tensors. FlashAttention's IO-complexity analysis is the reference quantification of why this is slow: the N×N intermediates dominate the HBM read/write volume, and because HBM bandwidth is the scarce resource, the kernel's runtime is set by bytes moved rather than by arithmetic [3].

Roofline analysis makes the target precise. With the H100's roughly 3.35 TB/s of HBM3 bandwidth and its dense FP16 tensor-core peak of about 989 TFLOP/s, the machine balance is about 295 FLOPs per byte (about 20 for non-tensor FP32 work); the eager scorer's arithmetic intensity is far below that balance, so it sits deep in the memory-bound region of the roofline, achieving only a small fraction of peak compute [16] [6]. The measured baseline for the project is the eager model's achieved memory throughput — profiled with Nsight Compute at roughly 40% of peak DRAM bandwidth for the serving shape (archived at the same batch as the C3 acceptance profile, §7.3) — which is the 40% figure that Objective O3 raises to 65% [5]. Deliverable B1 is the profiled baseline report: byte counts, achieved bandwidth, and throughput for the eager spatial scorer at production batch and candidate sizes.

6.2 Algebraic reformulation: turning distance computation into tensor-core GEMM

The kernel's central trick is algebraic. Expanding the negative squared Euclidean distance yields $-\|q - k\|^2 = -\|q\|^2 + 2(q \cdot k) - \|k\|^2$, an identity used classically in GPU similarity search to convert pairwise distance evaluation into a matrix multiplication plus precomputed row norms [29]. The rewrite matters because the elementwise difference-and-square formulation runs on general-purpose CUDA cores, while the $q \cdot k$ inner product is a dense GEMM that runs on the H100's fourth-generation tensor cores at an order of magnitude higher throughput [16]. The query norms $\|q\|^2$ are computed once per query, the key norms $\|k\|^2$ once per batch and head (inside the kernel, or by a small precompute kernel when several query tiles share them; in the Transformer the keys are per-request projections, not index vectors), and the inner loop reduces to GEMM, two broadcast subtractions, and the softmax accumulation.

$$-\|q - k\|^2 \;=\; -\|q\|^2 \;+\; 2\,(q \cdot k) \;-\; \|k\|^2$$

This reformulation is what allows a single fused kernel to express the entire scoring operation with tile-level reuse: a BLOCK_Q tile of queries and a BLOCK_K tile of candidates are staged in SRAM, multiplied with tl.dot so the Triton compiler emits tensor-core MMA instructions, and combined with the norms in registers before the next tile arrives [3] [26]. The exp/normalize structure is handled with the online-softmax running maximum and running sum accumulators introduced for attention kernels, so no N×N score matrix is ever written to memory [3] [25]. Deliverable B2 is the numerics-certified kernel: outputs matching the eager reference within rtol 1e-3, with unit tests over edge shapes (non-multiple-of-block candidate counts, masked tails) enforced via Triton's masked load/store discipline [26] [32].

6.3 Tiled IO-aware execution and the HBM traffic budget

The kernel's memory discipline follows the IO-aware design principle: read each input exactly once, keep every intermediate in the fastest storage that will hold it, and write each output exactly once [3]. The H100 memory hierarchy gives the concrete budget — 256 KB of register file and up to 228 KB of shared memory per SM, a 50 MB L2, and 80 GB of HBM3 at 3.35 TB/s — and the tile sizes BLOCK_Q, BLOCK_K, and BLOCK_D are chosen so that

the live tiles fit in shared memory with double-buffered prefetch through the L2 [16] [33]. Query norms and the online-softmax accumulators live in registers for the whole traversal of the candidate axis; only the final normalized scores are stored back to HBM [3].

Relative to the eager baseline, the byte accounting is direct. The eager pipeline moves the N×N difference matrix, its square, the reduced distances, the exponentiated scores, and the softmax intermediates to and from HBM — a multiple of the fused kernel's traffic, which moves only the inputs (Q, K, V, the per-head scale and the key-padding bias) and the final outputs once each. The project's budgeted outcome, a **40% reduction in bytes moved per query**, is therefore conservative against published fused-kernel results, where eliminating N×N materialization removes the dominant term entirely [3] [25]. At the serving shape (64 requests of 65 tokens, 4 heads of width 32, FP16) the traffic model puts the fused kernel at about 4.3 MB against about 303 MB for the eager chain: 1.4% of the bytes. The kernel's arithmetic intensity rises from about 0.6 to about 34 FLOPs per byte, still well below the H100's ridge point of about 295, so the fused scorer remains bandwidth-bound; the budgeted **3× inference throughput** uplift over eager PyTorch follows from removing about 98% of the bytes and the launch gaps between eager kernels, with the tensor-core GEMM taking over the distance arithmetic the SIMT pipeline used to do [29] [16]. Deliverable B3 is the acceptance run of Objective O2: Nsight Compute byte counters showing ≤ 60% of baseline bytes per query and a serving throughput measurement showing ≥ 3.0× the eager baseline at matched numerics [5].

6.4 Why Triton rather than hand-written CUDA

The kernel is authored in OpenAI's Triton rather than raw CUDA for two reasons. Triton is a Python-embedded domain-specific language in which the programmer reasons at the level of tiles and blocks while the compiler handles thread-level details — memory coalescing, shared-memory swizzling, and MMA emission — that dominate CUDA development time [26] [32]. Empirically, Triton kernels reach performance competitive with vendor libraries on the exact class of tiled, fusion-heavy workloads this project needs, which is why the PyTorch 2.0 compiler stack itself generates Triton for GPU codegen [26] [24]. The second reason is integration: because the model ships through torch.compile, a Triton kernel registers as a custom PyTorch operator and slots into the compiler pass of Workstream C without foreign-function glue [24] [10]. Deliverable B4 is the operator registration package: the kernel wrapped as a torch custom op with a fake/meta kernel for tracing, ready for pattern substitution.

7. Workstream C — Automated Compiler Pass for Kernel Substitution

7.1 Graph capture with TorchDynamo and torch.fx

The deployment path compiles the pricing model with torch.compile. TorchDynamo, the front end introduced with PyTorch 2.0, hooks CPython's frame evaluation API (PEP 523) to intercept Python bytecode before execution, symbolically traces the PyTorch operations it

finds, and lifts them into a torch.fx GraphModule — a pure dataflow representation in which the whole forward pass is a list of typed nodes with opcodes such as placeholder, call_function, and output [24] [9] [34]. torch.fx was designed for exactly this use: a minimal Python-to-Python IR that makes program analysis and rewriting accessible to non-compiler engineers, with pattern-based transformation as a first-class workflow [9].

Data-dependent control flow is the one structural hazard. Symbolic capture cannot trace branches whose condition depends on tensor values; TorchDynamo responds by cutting a graph break, compiling the pieces it has, and resuming eagerly — and a graph break inside the spatial-scoring region would hide the pattern the pass needs to find [24]. The modeling code therefore expresses its one dynamic branch (radius-dependent candidate trimming) with torch.cond, the higher-order operator that captures both branches into the graph as traceable subgraphs [35]. Deliverable C1 is the clean-capture certification: the production model compiles with zero graph breaks, verified by TorchDynamo's explanation utilities in CI [24].

7.2 Pattern definition and replacement via subgraph rewriting

The compiler pass is built on torch.fx.subgraph_rewriter.replace_pattern_with_filters (replace_pattern with semantic match filters), the API that matches a template subgraph against a GraphModule by use-def topology rather than by variable names, and splices a replacement subgraph into every non-overlapping match [10]. The pattern function reproduces, in ordinary PyTorch, the exact eager operator chain of the spatial scorer — subtract, square, sum over the feature axis, negate-scale, exponentiate, softmax-normalize — and is captured, through the same TorchInductor front end that compiles the model, into a template graph whose inputs act as wildcards [10]. The replacement function has the identical input signature and contains a single call to the custom operator wrapping the fused Triton kernel from Deliverable B4.

The pass is registered as TorchInductor's joint-graph pre-pass (joint_custom_pre_pass), the first hook that sees the normalized, decomposed ATen graph TorchInductor consumes and the stage at which Inductor runs its own attention fusion; by the post-grad hook, Inductor's pattern passes have already rewritten softmax with a temperature into a form that depends on how the negation and the coefficient were written [24]. During compilation the pass manager walks the captured graph, finds every topological match of the $O(N^2)$ eager distance pattern however a researcher spelled, structured or named it (exact sign and square canonicalization included; algebraically different formulations are separate catalog entries), excises the matched nodes, and wires the existing data dependencies into the fused-op node [9] [10]. The developer experience is then completely transparent: data scientists write idiomatic PyTorch, training and evaluation run unchanged in eager mode, and only the deployment compile rewrites the graph — the same philosophy that motivated the PyTorch 2.0 stack's design of preserving the eager programming model while harvesting graph-level optimizations at run time [24] [36]. Deliverable C2 is the pass itself, with a test matrix proving (a) match on canonical patterns, (b)

no match on semantically different lookalikes, and (c) end-to-end numerical equivalence of the compiled model against the eager reference.

7.3 Raising achieved memory bandwidth utilization from 40% to 65%

Objective O3's headline number is an efficiency metric, not a speed metric: the fraction of the H100's peak DRAM bandwidth that the serving kernel actually achieves, as reported by Nsight Compute's memory throughput counters [5]. The 40% baseline reflects the eager pipeline's stop-and-go access pattern — many small kernels, each underutilizing the memory system, with gaps for launch overhead and synchronization between them [3] [8]. After substitution, the single fused kernel streams data at full rate: tile prefetch keeps outstanding loads saturating the memory pipeline, tensor-core GEMM consumes bytes as fast as they arrive, and the elimination of N×N round trips removes the idle phases between bursts [3] [16].

The 65% target is set with published practice in mind rather than as an aspiration. Roofline methodology predicts that a fused, tile-scheduled kernel's achieved bandwidth approaches the practical ceiling of the streaming access pattern, and FlashAttention-class kernels demonstrate that fused attention-family workloads reach the compute-bound region of the roofline on A100/H100-class hardware at long sequence lengths (at Vectra's 65-token serving shape the RBF kernel stays bandwidth-bound, §6.3, which is why achieved DRAM bandwidth is the right efficiency metric) [3] [6]; GPU similarity-search kernels likewise report operating at large fractions of theoretical peak once k-selection and distance evaluation are fused and tiled [29]. The gap between the 65% target and the hardware's streaming ceiling is deliberate headroom for the serving environment's real conditions — concurrent model layers, KV traffic, and PCIe-attached candidate fetches. Deliverable C3 is the acceptance profile of Objective O3: a production-shape serving run at a batch of 1,024 requests, so that the working set exceeds the 50 MB L2 and the counters measure HBM streaming rather than launch latency, under Nsight Compute showing ≥ 65% of peak DRAM throughput for the substituted kernel region, against the archived 40% baseline profile from Deliverable B1 [5].

8. Project Timeline and Milestones

Execution runs as three overlapping workstreams across the January 6 – May 30, 2025 window. Workstream A fronts the schedule because the retrieval service carries external integration risk; Workstream B starts after the baseline profiler exists; Workstream C starts once the kernel's numerics are certified, and the final month is reserved for end-to-end acceptance under production load.

Phase	Dates	Workstream	Milestone and exit criteria
P0	Jan 6 – Jan 17, 2025	All	Requirements freeze; production query trace capture; managed-service baseline benchmark archived (QPS ceiling, latency profile, scatter-gather overhead) [1] [13]

P1	Jan 20 – Feb 14, 2025	A	A1 transport: async gRPC server and channel pool at 50k no-op QPS, p99 dispatch < 100 µs [19] [20]
P1	Jan 20 – Feb 7, 2025	B	B1 baseline: eager scorer profiled; 40% achieved-bandwidth figure archived [5]
P2	Feb 17 – Mar 14, 2025	A	A2 sharding: GeoHash routing table, adjacent-cell expansion, recall@10 ≥ 0.99 correctness suite [17] [18]
P2	Feb 10 – Mar 21, 2025	B	B2 kernel v1: Euclidean-expansion GEMM kernel numerically certified (rtol 1e-3) [29] [26]
P3	Mar 17 – Apr 11, 2025	A	A3 index: flat-memory HNSW + PQ shard; cluster load test at 50k QPS, p50 < 1 ms, recall@10 ≥ 0.95 — **O1 certified** [21] [22] [23]
P3	Mar 24 – Apr 25, 2025	B	B3 kernel v2: IO-aware tiling tuned; ≤ 60% baseline bytes/query, ≥ 3× throughput — **O2 certified** [3] [25]
P3	Apr 7 – Apr 25, 2025	C	C1 capture: zero-graph-break model compile with torch.cond refactor [24] [35]
P4	Apr 28 – May 16, 2025	C	C2 pass: subgraph rewriter live on the joint-graph pre-pass; match/no-match/equivalence test matrix green [9] [10]
P4	May 19 – May 30, 2025	C	C3 acceptance: serving profile ≥ 65% of peak DRAM throughput — **O3 certified**; end-to-end soak; production cutover from managed backend [6] [5]

The schedule embeds two deliberate slack points. The two-week overlap between A3 and B3 lets the retrieval soak and the kernel tuning proceed in parallel without coupling their acceptance runs, and the final two-week P4 window is reserved exclusively for compiler acceptance and cutover so that a late compiler defect cannot compress the production migration. If B3 slips, C2's test matrix still proceeds against kernel v1, because pattern substitution depends only on the operator interface, not on the tuned tile schedule.

9. Benchmarking and Verification Methodology

Retrieval metrics follow the community-standard ANN evaluation protocol: recall@10 against exact brute-force ground truth, latency percentiles measured at the client under an open-loop load generator (fixed arrival schedule, each latency measured from the request's scheduled send time, so that a stalled server cannot hide its own queueing), and QPS reported as the maximum sustained rate at which the latency bound holds [4] [7]. The corpus is the production 10M-property embedding set; the query workload is a replay of production traces captured in P0, which keeps the certified numbers honest against real spatial skew rather than uniform synthetic traffic. The cluster-level test drives 50,000 QPS for 30 minutes and records p50, p99,

and recall continuously; the acceptance threshold is p50 < 1 ms with recall@10 ≥ 0.95, and any disk spill or routing fan-out anomaly fails the run by construction [1] [4].

GPU metrics are measured, not modeled. Bytes moved per query come from Nsight Compute's dram__bytes_read and dram__bytes_write counters averaged over the production scoring shape (profiled with L2 flushed between kernel replays, at batches of 64 and 1,024); achieved-versus-peak bandwidth is the profiler's memory-throughput percentage against the H100's 3.35 TB/s peak [16] [5]. Throughput is measured in the serving stack — full model, production batching — rather than in kernel microbenchmarks, so the 3× figure is the throughput the product actually ships; numerics are gated by allclose at rtol 1e-3 on a fixed regression corpus before any performance number is accepted [3]. Roofline positions of the baseline and fused kernels are plotted from the same profile data so the improvement is legible as a move up the bandwidth roof toward the ridge point (the fused kernel remains left of the ridge at the serving shape) [6].

Compiler acceptance has three independent legs: structural (zero graph breaks; the pattern matcher fires exactly where expected and nowhere else, checked on a corpus of model variants), numerical (compiled model outputs equal eager outputs on the regression corpus), and performance (the C3 profile) [9] [10] [5]. Every acceptance artifact — raw counter dumps, load-test logs, recall reports — is archived with the run configuration so the three headline metrics remain auditable after the project closes.

10. Risk Register and Mitigations

Risk	Likelihood	Impact	Mitigation
GeoHash boundary effects erode recall under skewed radii	Medium	High	Adjacent-cell duplication bounded at nine shards; brute-force recall suite gates A2 [17] [18]
HTTP/2 stream limits silently queue requests at peak	Medium	High	Pre-warmed channel pool with round-robin selection; stream-occupancy telemetry alarm from P1 [27] [20]
HNSW memory blow-up at high M threatens RAM residency	Medium	Medium	PQ compression budget per shard; ef_search tuned on the recall-latency curve per ANN-Benchmarks protocol [4] [22]
Triton kernel regressions on non-standard shapes	Medium	High	Masked-load edge-shape unit tests; numerics gate precedes every performance acceptance [26] [32]
Graph breaks hide the pattern from the rewriter	Low	High	torch.cond refactor enforced by zero-break CI check (C1); torch.cond is a prototype feature, so the check reruns on every PyTorch upgrade [24] [35]

Pattern matcher misfires on lookalike code	Low	Medium	Negative test matrix; match filter on op targets, not names [10]
Profile overhead distorts the 65% acceptance measurement	Low	Medium	Fixed profiler configuration archived with B1; acceptance compares like-for-like profiles [5]

Two risks deserve narrative emphasis because they shape the design rather than merely the schedule. The first is tail amplification: every architectural decision in Workstream A — single-shard routing, thread-per-core pinning, arena allocation — is justified by the tail-at-scale argument that variance, not mean cost, is what a 50,000 QPS service actually fights [13]. The second is the numerics-versus-performance coupling in Workstream B: fused kernels historically trade accuracy for speed through sloppy reduction order, so the project's rtol 1e-3 gate is deliberately stricter than the pricing model's own tolerance, ensuring the compiler pass can never silently degrade model quality when it substitutes the fast path [3] [9].

11. Team, Resources, and Deliverables

The project staffs one engineer per workstream with shared infrastructure. Workstream A requires the bare-metal retrieval cluster (three geographic macro-regions, N+1 nodes per region sized for the PQ-compressed index), Workstream B requires a dedicated H100 development host plus profiler access, and Workstream C requires CI capacity for the compile-and-compare test matrix. Hardware sizing follows from the capacity model: PQ codes (16 bytes per 128-dimensional embedding) drive traversal, while the FP32 vectors kept for the exact re-rank, the graph and the property columns bring the index to about 785 bytes per vector, roughly 7.9 GB for the 10M corpus or about 2.6 GB per shard across three shards, which leaves most of each node's RAM free [22] [23].

The deliverable set is exactly the acceptance chain: A1–A3 certify Objective O1; B1–B4 certify Objective O2 and arm Workstream C; C1–C3 certify Objective O3. Documentation deliverables accompany each code deliverable — the routing-table specification, the kernel's byte-traffic budget sheet, and the pass's pattern catalog — so that the system remains operable and extensible after the project window closes on May 30, 2025.

12. Works Cited

[1] J. J. Pan, J. Wang, and G. Li, "Survey of Vector Database Management Systems," *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024. Available: https://arxiv.org/abs/2310.14021

[2] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.

[4] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms," *Information Systems*, vol. 87, 2020. Available: https://arxiv.org/abs/1807.05614

[5] NVIDIA Corporation, "Nsight Compute Documentation — Kernel Profiling and Memory Workload Analysis," NVIDIA Developer Documentation, 2019–2024. Available: https://docs.nvidia.com/nsight-compute/

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[7] H. V. Simhadri et al., "Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search," in *Proc. NeurIPS 2021 Competitions and Demonstrations Track*, PMLR vol. 176, 2022, pp. 177–189.

[8] R. Pope et al., "Efficiently Scaling Transformer Inference," in *Proc. 6th Conf. on Machine Learning and Systems (MLSys)*, 2023.

[9] J. Reed et al., "torch.fx: Practical Program Capture and Transformation for Deep Learning in Python," in *Proc. 5th Conf. on Machine Learning and Systems (MLSys)*, 2022. Available: https://arxiv.org/abs/2112.08429

[10] PyTorch Documentation, "torch.fx.subgraph_rewriter.replace_pattern," PyTorch 1.13+ API Reference, 2022–2024. Available: https://pytorch.org/docs/stable/fx.html

[11] J. Wang et al., "Milvus: A Purpose-Built Vector Data Management System," in *Proc. 2021 Int. Conf. on Management of Data (SIGMOD)*, 2021, pp. 2614–2627.

[12] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[13] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[14] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, "Transformer Dissection: An Unified Understanding of Transformer's Attention via the Lens of Kernel," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 4344–4353. Available: https://arxiv.org/abs/1908.11775

[15] J. T. Frank, S. Chmiela, K.-R. Müller, and O. T. Unke, "Euclidean Fast Attention — Machine Learning Global Atomic Representations at Linear Cost," arXiv:2412.08541, Dec. 2024. Available: https://arxiv.org/abs/2412.08541

[16] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture" (whitepaper) and "NVIDIA Hopper Architecture In-Depth," NVIDIA Technical Blog, Mar. 2022. Available: https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

[17] "Geohash," Wikipedia, The Free Encyclopedia (encoding by G. Niemeyer, 2008). Available: https://en.wikipedia.org/wiki/Geohash

[18] Elastic, "Geohash Grid Aggregation," *Elasticsearch Reference* (7.x). Available: https://www.elastic.co/guide/en/elasticsearch/reference/7.17/search-aggregations-bucket-geohashgrid-aggregation.html

[19] gRPC Authors, "Asynchronous-API tutorial — C++," gRPC Documentation, 2022. Available: https://grpc.io/docs/languages/cpp/async/

[20] M. Belshe, R. Peon, and M. Thomson, "Hypertext Transfer Protocol Version 2 (HTTP/2)," IETF RFC 7540, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7540

[21] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, Apr. 2020. Available: https://arxiv.org/abs/1603.09320

[22] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, Jan. 2011.

[23] Y. Malkov et al., "hnswlib: Header-only C++/Python Library for Fast Approximate Nearest Neighbors," GitHub, 2017–2024. Available: https://github.com/nmslib/hnswlib

[24] J. Ansel et al., "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," in *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, vol. 2, 2024, pp. 929–947. Available: https://dl.acm.org/doi/10.1145/3620665.3640366

[25] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024. Available: https://arxiv.org/abs/2307.08691

[26] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.

[27] gRPC Authors, "Performance Best Practices," gRPC Documentation. Available: https://grpc.io/docs/guides/performance/

[28] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 1964–1978, 2021. Available: https://arxiv.org/abs/2101.12631

[29] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. Available: https://arxiv.org/abs/1702.08734

[30] C. Aguerrebere, I. Bhati, M. Hildebrand, M. Tepper, and T. Willke, "Similarity Search in the Blink of an Eye with Compressed Indices," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3433–3446, 2023.

[31] R. Guo et al., "Accelerating Large-Scale Inference with Anisotropic Vector Quantization," in *Proc. 37th Int. Conf. on Machine Learning (ICML)*, 2020.

[32] P. Tillet, "Introducing Triton: Open-Source GPU Programming for Neural Networks," OpenAI Blog, Jul. 2021. Available: https://openai.com/index/triton/

[33] NVIDIA Corporation, "CUDA C++ Programming Guide — Hopper Tuning Guide," NVIDIA Developer Documentation, 2022–2024. Available: https://docs.nvidia.com/cuda/hopper-tuning-guide/

[34] B. Cannon and D. Viehland, "PEP 523 — Adding a Frame Evaluation API to CPython," Python Enhancement Proposals, May 2016. Available: https://peps.python.org/pep-0523/

[35] PyTorch Documentation, "torch.cond — Higher-Order Operator for Data-Dependent Control Flow," PyTorch 2.1+ Documentation, 2023. Available: https://pytorch.org/docs/stable/cond.html

[36] PyTorch Foundation, "PyTorch 2.0: Our Next Generation Release That Is Faster, More Pythonic and Dynamic as Ever," PyTorch Blog, Mar. 2023. Available: https://pytorch.org/blog/pytorch-2.0-release/